QueryLang: A Custom Domain-Specific Language and Compiler Engine for Tabular Data Analytics

Name: Yuktha Sreenivasa Anjaneyulu
RegNo: 23BDS0287

1. Abstract

In modern data science and analytics, filtering, projecting, and aggregating tabular text files (such as CSVs) typically requires writing verbose scripts in general-purpose languages like Python or setting up heavy database management systems (DBMS) like PostgreSQL or MySQL. This project, QueryLang, introduces a custom Domain-Specific Language (DSL) that allows users to perform SQL-like operations directly on flat files.

From a compiler design perspective, QueryLang implements a full compilation pipeline—comprising a Flex-based Lexer, a Bison-based LALR(1) Parser, a Symbol Table manager for schema tracking, a Semantic Analysis engine for type and column validation, and an Interpretation/Execution backend that processes the query against a target dataset. This report outlines the Phase 1 system design, language specification, and architectural blueprint required to build and deploy this system.

2. Problem Statement

Handling tabular text files efficiently without deploying heavy database infrastructure remains a recurring challenge for data analysts. Existing general-purpose tools require boilerplate code just to open a file, parse lines, handle missing data, and perform calculations. Conversely, setting up full relational databases introduces unnecessary overhead for quick data transformations.

The problem addressed by this project is the design and implementation of a custom compiler for a mini-SQL query language. The compiler must intercept, parse, validate, and execute user-defined data queries while ensuring that lexical errors, syntax errors, and semantic schema inconsistencies (such as referencing non-existent columns or performing invalid type operations) are caught at compile-time with meaningful error diagnostics.

3. Motivation

While standard compilers process programming languages into machine code, domain-specific language compilers bridge the gap between human intent and data processing. The primary motivations for this project are:

- Practical Application of Theory: To bridge theoretical compiler concepts—such as parse trees, symbol tables, and semantic validation—with practical data science utilities.
- Performance & Lightweight Footprint: To create an ultra-lightweight query processor that runs directly on raw text files without heavy runtime libraries.
- Educational Value: To master the inner workings of language translation, error recovery strategies (yyerror), and intermediate execution environments.

4. Project Objectives

1. Language Design: Define a clean, expressive, SQL-like grammar syntax for loading, selecting, filtering, and aggregating tabular data.
2. Lexical & Syntax Analysis: Build a robust tokenization and parsing system using Flex and Bison to validate script structures.
3. Symbol Table Management: Implement data structures to dynamically track dataset schemas, column headers, and inferred data types (INTEGER, FLOAT, STRING).
4. Semantic Verification: Implement compile-time checks to verify that queried columns exist in the target dataset and that conditional expressions are type-safe.
5. Execution Engine: Develop an interpretation backend that executes the validated Abstract Syntax Tree (AST) on actual CSV datasets and prints formatted analytics.
6. Error Handling: Implement strict error recovery and messaging routines using custom yyerror() functions for lexical, syntax, and semantic errors.

5. Scope of the Project

- **In-Scope:**
  - Loading single CSV datasets via file paths.
  - Column projection (SELECT col1, col2).
  - Conditional row filtering (FILTER col > value).
  - Basic aggregations (SUM, AVG, COUNT, MAX, MIN).
  - Compile-time schema checking and symbol table lookups.
  - Console-based output generation.
- **Out-of-Scope (Future Enhancements):**
  - Multi-file inner/outer SQL joins.
  - Persistent database storage (modifying the underlying source CSV file).
  - Distributed query optimization over computer clusters.

6. Background Study and Literature Survey

Traditional text processing relies heavily on Unix utilities like awk, sed, and grep, which lack type safety and structured syntax verification. Modern data analysis frameworks like Pandas (Python) or dplyr (R) provide powerful data manipulation but act as software libraries rather than dedicated, compiled domain languages.

Compiler construction tools such as Lex/Flex (lexical analyzer generator) and Yacc/Bison (parser generator) provide the theoretical foundation for building language processors. By leveraging LALR(1) parsing algorithms, we can guarantee deterministic syntax evaluation and immediate detection of malformed statements, making them ideal for building custom data query parsers.

7. Compiler Design Concepts Involved

The development of QueryLang incorporates almost every major pillar of compiler design:

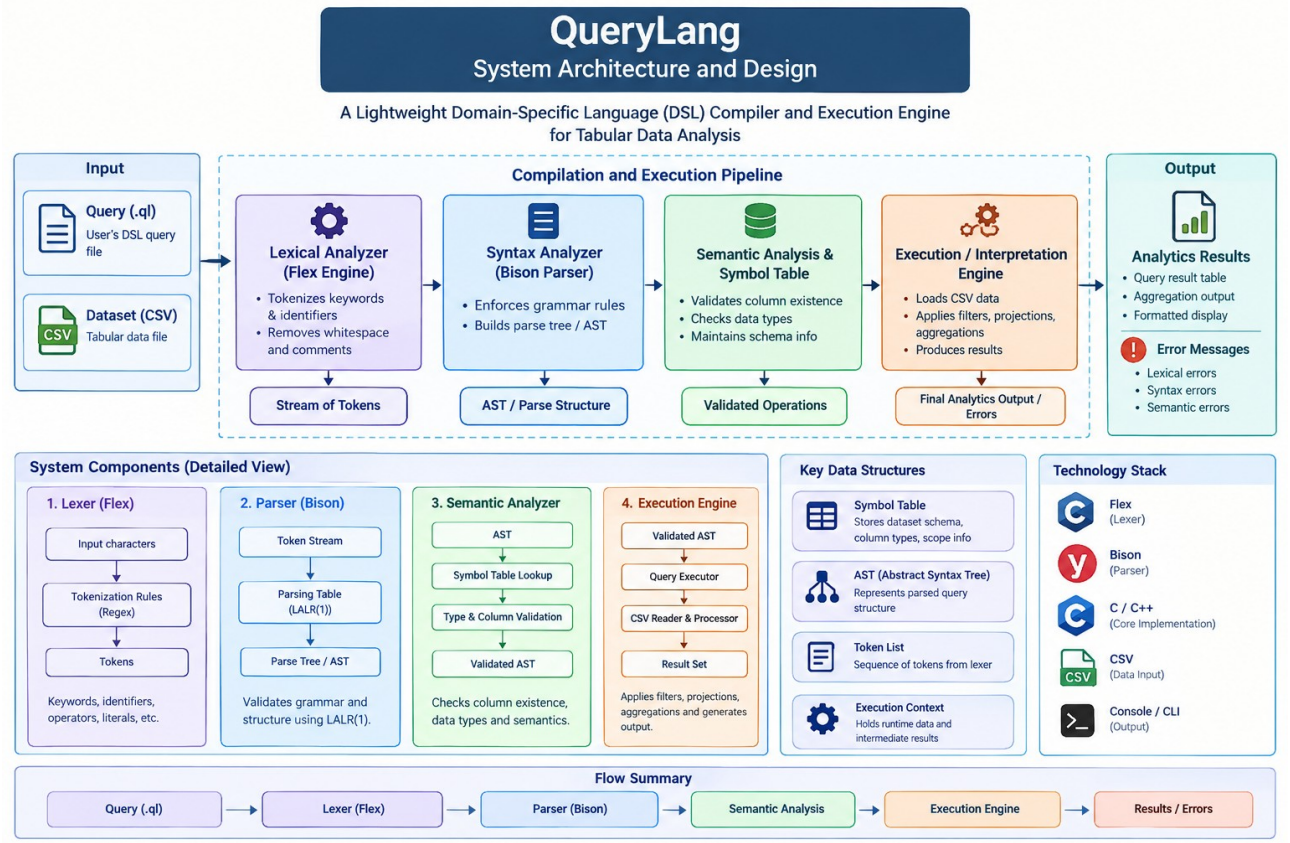
1. Lexical Analysis: Scans source code characters and groups them into meaningful tokens (keywords, identifiers, literals, operators).
2. Syntax Analysis (Parsing): Uses context-free grammar (CFG) rules to build parse trees and ensure structural correctness.
3. Symbol Table Management: Stores metadata regarding dataset attributes, column names, and data types to maintain scope and state.
4. Semantic Analysis: Performs context-dependent checks, verifying that referenced attributes exist in the symbol table and checking data type compatibility.
5. Error Detection and Recovery: Gracefully catches syntax and semantic violations and outputs descriptive failure diagnostics without crashing abruptly.
6. Interpretation Engine: Traverses the parsed structures to evaluate expressions and compute analytics directly on target files.

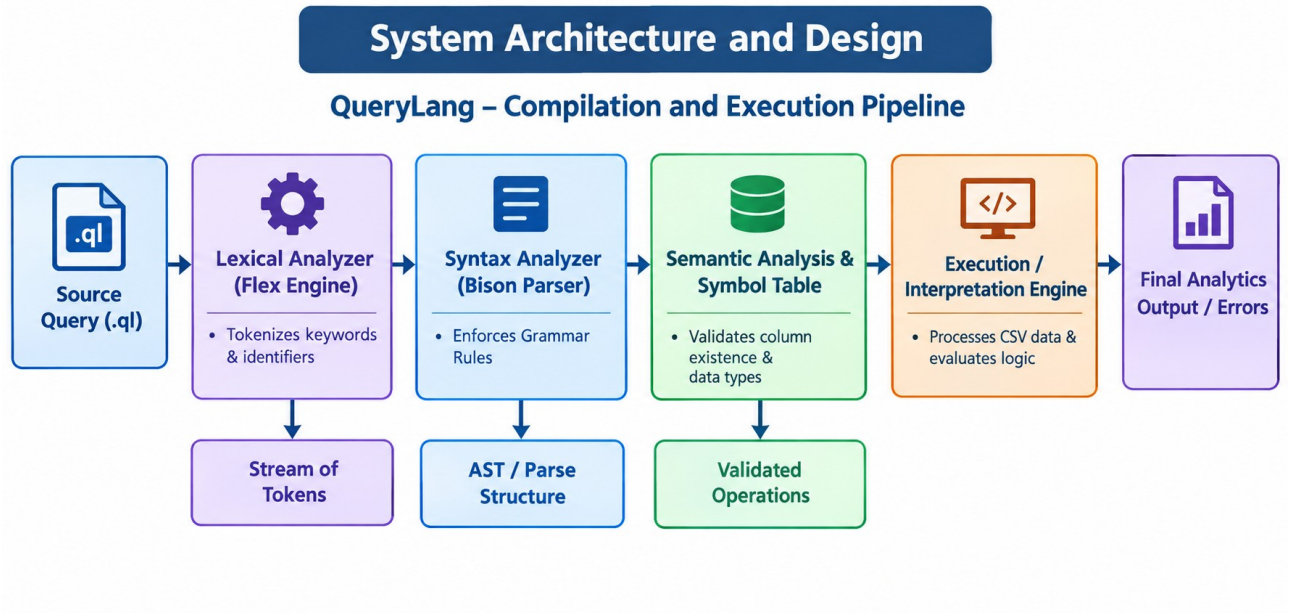
8. Proposed Methodology and System Workflow

The compilation and execution pipeline of QueryLang follows a sequential, multi-stage model:

1. Source Input: The user writes a script in the QueryLang file format (e.g., query.ql).
2. Lexical Phase (Flex): The source text is broken down into tokens (LOAD, SELECT, identifiers, string literals, numeric constants).
3. Syntax Phase (Bison): Tokens are matched against grammar production rules. If a mismatch occurs, the parser triggers yyerror() and reports an "Invalid Query Structure".
4. Semantic & Symbol Table Phase: The parser reads the header row of the target CSV file specified in the LOAD command, populating the Symbol Table with column names and types. Subsequent SELECT and FILTER tokens are validated against this table.
5. Execution Engine: If all compilation and semantic checks pass successfully, the interpreter executes the operations row-by-row on the dataset and prints the results.

9. System Architecture and Design





10. Language Specification (Syntax & Tokens)

Keywords & Operators

- Keywords: LOAD, AS, SELECT, FILTER, AGGREGATE
- Operators: =, ==, >, <, >=, <=, !=, ,, ;

- Identifiers: Alphanumeric strings representing column names or variable aliases (e.g., salary, age, name).
- Literals: Integer values, floating-point numbers, and double-quoted strings (file paths or text filters).

Sample QueryLang Script Syntax

SQL

```
LOAD "employees.csv" AS emp_data; SELECT
name, department, salary; FILTER salary >
60000;
AGGREGATE AVG(salary);
```

Context-Free Grammar (CFG) Outline

Plaintext

```
program        : load_stmt select_stmt filter_stmt agg_stmt ';'
               ;
load_stmt      : LOAD STRING AS IDENTIFIER
               ;
select_stmt    : SELECT column_list
               ;
column_list    : IDENTIFIER
               | IDENTIFIER ',' column_list
               ;
filter_stmt    : FILTER condition
               | /* epsilon */
               ;
condition      : IDENTIFIER COMP_OP NUMBER
               ;
agg_stmt       : AGGREGATE FUNC_NAME '(' IDENTIFIER ')'
               | /* epsilon */
               ;
```

11. Technology Stack

- Programming Language: C (for core data structures, symbol table implementation, and execution engine logic).
- Compiler Tools:
  - Flex (Fast Lexical Analyzer Generator)
  - Bison (GNU Parser Generator)
- Build Automation: Make utility (gcc, make)
- Development Environment: Linux (Ubuntu) / Windows Subsystem for Linux (WSL) / Visual Studio Code.

12. Initial Working Prototype (Phase 1 Baseline)

Initial prototype skeleton demonstrating the core scanner (query.l) and parser (query.y) integration.

Lexer Specification (query.l)

Code snippet

```
%{
#include "y.tab.h"
#include <string.h>
%}

%%

"LOAD"          { return LOAD; }
"AS"            { return AS; } "SELECT"
                { return SELECT; } "FILTER"
                { return FILTER; }
"AGGREGATE" { return AGGREGATE; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.sval = strdup(yytext); return IDENTIFIER; }
\"[^\"]*\"   { yylval.sval = strdup(yytext); return STRING; } [0-9]+        { yylval.ival =
atoi(yytext); return NUMBER; } [>,<=\+\-;] { return yytext[0]; }
[ \t\n\r]           { /* ignore whitespace */ }
.                   { printf("Lexical Error: Unknown character '%s'\n", yytext); }

%%

int yywrap(void) { return 1; }
```

Parser Specification (query.y)

Code snippet

```
%{
#include <stdio.h> #include
<stdlib.h> #include <string.h>

void yyerror(const char *s); int yylex(void);
%}

%union {
        char *sval; int ival;
}

%token LOAD AS SELECT FILTER AGGREGATE
%token <sval> IDENTIFIER STRING
%token <ival> NUMBER

%%

program:
        LOAD STRING AS IDENTIFIER ';' SELECT column_list ';' {
                printf("[Phase 1 Prototype Success]: Query structure parsed successfully!\n");
                printf("Target Dataset File: %s\n", $2);
        }
;
```

```
column_list:
        IDENTIFIER
        | IDENTIFIER ',' column_list
;

%%

void yyerror(const char *s) { fprintf(stderr, "Syntax Error: %s\n",
        s);
}
int main(void) {
        printf("Enter Query Statement:\n"); yyparse();
        return 0;
}
```

13. FUNCTIONAL REQUIREMENTS

FR-01: The system shall accept QueryLang source files with the .ql extension.

FR-02: The system shall tokenize keywords, identifiers, literals, operators, and delimiters using Flex.

FR-03: The system shall validate the query structure using the Bison LALR(1) parser.

FR-04: The system shall maintain a symbol table containing CSV column names and data types.

FR-05: The system shall validate column existence and data type compatibility.

FR-06: The system shall support SELECT, FILTER, and basic aggregation operations such as SUM, AVG, COUNT, MAX, and MIN.

FR-07: The system shall execute validated queries on CSV datasets and generate formatted analytics output.

FR-08: The system shall report lexical, syntax, semantic, and file-related errors.

14.NON-FUNCTIONAL REQUIREMENTS

NFR-01: The system shall provide fast and efficient processing of small to medium-sized CSV datasets.

NFR-02: The system shall provide clear and meaningful error messages for invalid queries and input data.

NFR-03: The compiler components shall be modular and maintainable, allowing individual stages to be modified independently.

NFR-04: The system shall provide reliable and consistent query execution for valid QueryLang programs.

NFR-05: The system shall be compatible with Linux/Ubuntu environments and standard C, Flex, and Bison tools.

NFR-06: The system shall maintain a simple command-line interface for ease of use and testing.

15.ERROR HANDLING

The system handles errors at different stages of the processing pipeline:

- Lexical Errors – Invalid or unrecognized characters.
- Syntax Errors – Incorrect query structure or grammar violations.
- Semantic Errors – Non-existent columns or invalid operations.
- Data Type Errors – Incompatible operations between different data types.
- File Errors – Missing or inaccessible CSV datasets.

Meaningful error messages are generated to help identify and correct invalid queries.

16. EXPECTED OUTCOMES

The project is expected to produce:

- A working QueryLang domain-specific language for basic tabular data analysis.
- A Flex-based lexical analyzer and Bison-based parser.
- An AST / parse structure for valid queries.
- A symbol table for CSV schema and data type information.
- Semantic validation for columns and operations.
- An execution engine for CSV-based data processing.
- Support for SELECT, FILTER, SUM, AVG, COUNT, MAX, and MIN.
- Meaningful error reporting and console-based analytics output.